TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
KHOA CÔNG NGHỆ VÀ KỸ THUẬT

LẬP TRÌNH PYTHON VÀ ỨNG DỤNG

HỆ THỐNG QUẢN LÝ PHÒNG GYM

Phan Trần Gia Hưng – 0024417689
LỚP: ĐHCNTT24B-IT

Đồng Tháp, tháng 4 năm 2026

TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
KHOA CÔNG NGHỆ VÀ KỸ THUẬT

LẬP TRÌNH PYTHON VÀ ỨNG DỤNG

HỆ THỐNG QUẢN LÝ PHÒNG GYM

(Gym Management System)

0024417689 – Phan Trần Gia Hưng

LỚP: ĐHCNTT24B-IT

GIẢNG VIÊN HƯỚNG DẪN: ThS. Nguyễn Quốc Anh

Đồng Tháp, tháng 4 năm 2026

Mục lục

1 Lý do chọn đề tài và Mục tiêu **1**
1.1 Lý do chọn đề tài 1
1.2 Mục tiêu đề tài 1

2 Mô tả bài toán **1**
2.1 Các chức năng chính 2

3 Tổng quan về Python **2**
3.1 Đặc điểm 2
3.2 Thư viện sử dụng 2

4 Cơ sở dữ liệu **3**
4.1 Các bảng dữ liệu 3
4.2 Sơ đồ quan hệ giữa các bảng 3
4.3 Chi tiết bảng members 4

5 Kết nối Python với CSDL **4**
5.1 Context Manager `get_db()` 4
5.2 Khởi tạo Database – `init_db()` 4
5.3 Migration 5

6 Cài đặt các chức năng chính **5**
6.1 Thêm (Create) 6
6.2 Xem (Read) 7
6.3 Sửa (Update) 7
6.4 Xóa (Delete) – Soft Delete 8

7 Một số đoạn code tiêu biểu **8**
7.1 Nhận diện khuôn mặt (Face ID) 8
7.2 BaseModel và Soft Delete Pattern 9
7.3 Navigation Pattern – Monkey-patch trên page 10
7.4 GUI State với Dict Closures 11
7.5 Đăng ký khuôn mặt (Face Registration) 11

1. Lý do chọn đề tài và Mục tiêu

1.1. Lý do chọn đề tài

Trong bối cảnh ngành fitness và gym đang phát triển mạnh mẽ tại Việt Nam, việc quản lý phòng gym theo cách truyền thống (sổ sách, Excel) gặp nhiều hạn chế:

- Khó theo dõi thông tin hội viên, gói tập, thiết bị khi số lượng lớn.
- Điểm danh thủ công tốn thời gian, dễ sai sót và không chuyên nghiệp.
- Không có công cụ thống kê, báo cáo để hỗ trợ ra quyết định kinh doanh.
- Hội viên không thể tự tra cứu thông tin gói tập, lịch sử của mình.

Đề tài này nhằm xây dựng một hệ thống quản lý phòng gym toàn diện, tích hợp công nghệ **nhận diện khuôn mặt (Face ID)** để tự động hóa điểm danh, nâng cao trải nghiệm người dùng và hiệu quả vận hành.

1.2. Mục tiêu đề tài

1. Xây dựng ứng dụng desktop quản lý phòng gym với đầy đủ chức năng CRUD cho hội viên, gói tập, thiết bị, huấn luyện viên.
2. Tích hợp nhận diện khuôn mặt để điểm danh tự động thông qua camera.
3. Cung cấp dashboard thống kê và báo cáo trực quan cho quản trị viên.
4. Xây dựng app riêng cho hội viên và huấn luyện viên tự tra cứu thông tin.
5. Áp dụng kiến trúc phân tầng (Layered Architecture) để dễ bảo trì và mở rộng.

2. Mô tả bài toán

Hệ thống gồm **2 ứng dụng riêng biệt** cùng chia sẻ một backend:

- **GymAdmin App** (`app/main.py`): Dành cho quản trị viên – quản lý toàn bộ hệ thống.
- **GymFit Member App** (`app/user_main.py`): Dành cho hội viên và huấn luyện viên – tự tra cứu thông tin cá nhân.

2.1. Các chức năng chính

Bảng 1: Danh sách chức năng hệ thống

Module	**Chức năng**	**App**
Quản lý hội viên	Thêm, sửa, xóa (mềm), tìm kiếm, lọc theo giới tính/trạng thái	Admin
Quản lý gói tập	CRUD gói tập, đăng ký, gia hạn, hủy, tự động hết hạn	Admin
Quản lý thiết bị	CRUD, lọc theo trạng thái (hoạt động/bảo trì/hỏng)	Admin
Quản lý HLV	CRUD huấn luyện viên, gán học viên	Admin
Điểm danh Face ID	Nhận diện khuôn mặt qua camera, check-in/check-out tự động	Admin
Dashboard	KPI, biểu đồ doanh thu, cảnh báo sắp hết hạn	Admin
Báo cáo	Thống kê hội viên, doanh thu, thiết bị	Admin
Xem thông tin cá nhân	Hội viên/HLV tự xem profile, gói tập, lịch sử	User
Thông báo	Cảnh báo gói sắp hết hạn, thông báo hệ thống	User

3. Tổng quan về Python

3.1. Đặc điểm

Python là ngôn ngữ lập trình bậc cao, được thiết kế với cú pháp đơn giản, dễ đọc. Các đặc điểm nổi bật:

- **Cú pháp sạch sẽ**, dễ học, phù hợp cho cả người mới và chuyên gia.
- **Thư viện phong phú**: Hỗ trợ từ GUI, machine learning, xử lý ảnh đến database.
- **Đa nền tảng**: Chạy trên Windows, macOS, Linux mà không cần sửa code.
- **Cộng đồng lớn**: Dễ tìm kiếm tài liệu, hỗ trợ và thư viện mã nguồn mở.

3.2. Thư viện sử dụng

Bảng 2: Các thư viện chính

Thư viện	**Phiên bản**	**Vai trò**
`flet`	0.82.2	Framework GUI desktop, xây dựng giao diện người dùng
`sqlite3`	Built-in	Cơ sở dữ liệu quan hệ dạng file, không cần server
`face_recognition`	1.3.0	Nhận diện và mã hóa khuôn mặt dựa trên dlib
`opencv-python`	4.x	Xử lý ảnh, chụp ảnh từ camera (cv2)
`numpy`	1.x	Tính toán ma trận, hỗ trợ face_recognition
`uuid`	Built-in	Tạo ID duy nhất dạng UUID4 cho mọi bản ghi

4. Cơ sở dữ liệu

Hệ thống sử dụng **SQLite3** – cơ sở dữ liệu dạng file, không cần cài đặt server riêng. File database: `data/gym_db.db`.

4.1. Các bảng dữ liệu

Hệ thống gồm **10 bảng**:

Bảng 3: Danh sách các bảng trong CSDL

STT	Tên bảng	Mô tả
1	`members`	Thông tin hội viên (tên, SĐT, email, PIN, Face ID)
2	`membership_plans`	Định nghĩa các gói tập (tên, giá, thời hạn)
3	`subscriptions`	Đăng ký gói tập (liên kết member ↔ plan)
4	`equipment`	Thiết bị phòng gym (tên, loại, trạng thái)
5	`trainers`	Huấn luyện viên (tên, chuyên môn, PIN)
6	`notifications`	Thông báo cho hội viên/HLV
7	`trainer_assignments`	Liên kết HLV ↔ học viên
8	`training_sessions`	Buổi tập HLV–học viên
9	`attendance`	Điểm danh (check-in/out bằng Face ID)
10	`transactions`	Giao dịch thanh toán

4.2. Sơ đồ quan hệ giữa các bảng

```
members --1:N--> subscriptions <--N:1-- membership_plans
   |                  |
   |     1:N          |
   |--------> attendance
   |--------> transactions
   |--------> trainer_assignments <--N:1-- trainers
   |                  |
   |                  |--> training_sessions
   |--------> notifications
```

Quy tắc chung:

- Tất cả ID là **UUID4** (TEXT PRIMARY KEY), không dùng auto-increment.
- Mọi bảng (trừ notifications) có `created_at`, `updated_at`, `is_active`.
- Foreign Keys được bật bằng `PRAGMA foreign_keys = ON`.
- Soft delete: `is_active = 0` thay vì DELETE.

4.3. Chi tiết bảng members

Bảng 4: Cấu trúc bảng `members`

Cột	Kiểu	Ràng buộc	Mô tả
id	TEXT	PK	UUID4 duy nhất
name	TEXT	NOT NULL	Họ tên hội viên
phone	TEXT	NOT NULL, UNIQUE	Số điện thoại (đăng nhập)
email	TEXT	–	Email
gender	TEXT	–	male/female/other
date_of_birth	TEXT	–	Ngày sinh (YYYY-MM-DD)
pin	TEXT	NOT NULL, CHECK(6)	Mã PIN 6 số
face_registered	INTEGER	DEFAULT 0	Đã đăng ký Face ID chưa
is_active	INTEGER	DEFAULT 1	Soft delete flag

5. Kết nối Python với CSDL

5.1. Context Manager `get_db()`

Hệ thống sử dụng Context Manager để quản lý kết nối database an toàn:

```python
@contextmanager
def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row      # Ket qua dang dict-like
    conn.execute("PRAGMA foreign_keys = ON")  # Bat Foreign Key
    try:
        yield conn          # Tra connection cho caller
        conn.commit()       # Khong loi -> commit
    except Exception:
        conn.rollback()     # Co loi -> rollback
        raise
    finally:
        conn.close()        # Luon dong connection
```

Listing 1: Context Manager kết nối database

Giải thích:

- `@contextmanager`: Biến hàm thành context manager, dùng với `with`.
- `row_factory = sqlite3.Row`: Truy cập cột bằng tên (`row["name"]`) thay vì index.
- `PRAGMA foreign_keys = ON`: SQLite mặc định tắt FK, phải bật thủ công.
- Tự động `commit` khi thành công, `rollback` khi lỗi, `close` luôn luôn.

5.2. Khởi tạo Database – `init_db()`

Hàm `init_db()` được gọi một lần khi app khởi động, tạo 10 bảng và các index:

Listing 2: Tạo bảng members

```sql
CREATE TABLE IF NOT EXISTS members (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    phone TEXT NOT NULL UNIQUE,
    email TEXT,
    gender TEXT,
    date_of_birth TEXT,
    address TEXT,
    emergency_contact TEXT,
    photo TEXT,
    pin TEXT NOT NULL DEFAULT '000000' CHECK(LENGTH(pin) = 6),
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    is_active INTEGER NOT NULL DEFAULT 1
);
```

5.3. Migration

Để thêm cột mới vào database cũ mà không mất dữ liệu, hệ thống dùng pattern `try/except` `ALTER TABLE`:

```python
# Them cot face_registered neu chua co
try:
    conn.execute(
        "ALTER TABLE members ADD COLUMN face_registered "
        "INTEGER NOT NULL DEFAULT 0"
    )
except Exception:
    pass  # Cot da ton tai -- bo qua
```

Listing 3: Migration thêm cột

6. Cài đặt các chức năng chính

Hệ thống tuân theo kiến trúc phân tầng nghiêm ngặt: **GUI** → **Service** → **Repository** → **Database**.

```
+-------------------------------------------------------------+
|                      GUI Layer (Flet)                       |
|  login | dashboard | members | memberships | equipment      |
|  trainers | reports | attendance | face_register           |
+----------------------------+-------------------------------+
                             |
+----------------------------v-------------------------------+
|                     Services Layer                          |
|  member_svc | membership_svc | equipment_svc                |
|  trainer_svc | attendance_svc | face_svc                    |
|  Validation -> Business Logic -> Goi Repository            |
+----------------------------+-------------------------------+
                             |
```

```
+---------------------------v-----------------------------+
|                Repositories Layer                        |
|  member_repo | membership_repo | equipment_repo          |
|  trainer_repo | attendance_repo                          |
|  SQL thuan (INSERT/SELECT/UPDATE) -> Model objects       |
+---------------------------+-----------------------------+
                            |
+---------------------------v-----------------------------+
|                  Models Layer                            |
|  BaseModel | Member | MembershipPlan | Equipment         |
|  Trainer | MembershipSubscription | Attendance           |
+---------------------------+-----------------------------+
                            |
+---------------------------v-----------------------------+
|                 Core Layer + Database                    |
|  config.py | database.py | security.py | SQLite3          |
+-------------------------------------------------------+
```

6.1. Thêm (Create)

```python
# app/services/member_svc.py
def create_member(name, phone, email=None, gender=None,
                  date_of_birth=None, address=None,
                  emergency_contact=None):
    # Buoc 1: Validate du lieu dau vao
    if not name or not name.strip():
        raise ValueError("Ten khong duoc de trong")
    if not phone or not re.match(r'^0\d{9}$', phone):
        raise ValueError("So dien thoai khong hop le")

    # Buoc 2: Tao model object
    member = Member(name=name.strip(), phone=phone.strip(),
                    email=email, gender=gender,
                    date_of_birth=date_of_birth,
                    address=address,
                    emergency_contact=emergency_contact)

    # Buoc 3: Goi repository de luu vao DB
    return member_repo.create(member)
```

Listing 4: Thêm hội viên mới – Service Layer

```python
# app/repositories/member_repo.py
def create(member):
    with get_db() as conn:
        conn.execute("""
            INSERT INTO members
            (id, name, phone, email, gender, date_of_birth,
             address, emergency_contact, photo, pin,
             created_at, updated_at, is_active)
            VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)
        """, (member.id, member.name, member.phone,
              member.email, member.gender,
```

```
                member.date_of_birth, member.address,
                member.emergency_contact, member.photo,
                member.pin,
                member.created_at.isoformat(),
                member.updated_at.isoformat(),
                int(member.is_active)))
    return member
```

Listing 5: Thêm hội viên – Repository Layer

6.2. Xem (Read)

```
# Repository: lay tat ca hoi vien dang active
def get_all(include_inactive=False):
    with get_db() as conn:
        if include_inactive:
            rows = conn.execute(
                "SELECT * FROM members ORDER BY created_at DESC"
            ).fetchall()
        else:
            rows = conn.execute(
                "SELECT * FROM members WHERE is_active = 1 "
                "ORDER BY created_at DESC"
            ).fetchall()
        return [_row_to_model(row) for row in rows]

# Chuyen row thanh object -- dung __new__ khong phai __init__
def _row_to_model(row):
    m = Member.__new__(Member)    # Khong goi __init__
    m.id = row["id"]              # Giu UUID tu DB
    m.name = row["name"]
    m.phone = row["phone"]
    m.created_at = datetime.fromisoformat(row["created_at"])
    m.is_active = bool(row["is_active"])  # 0/1 -> bool
    # ... cac field khac
    return m
```

Listing 6: Lấy danh sách hội viên

Tại sao dùng `__new__` thay vì `__init__`? Vì `Member()` sẽ tạo UUID mới và ghi đè giá trị từ database.

6.3. Sửa (Update)

```
# Service layer
def update_member(member_id, **kwargs):
    member = member_repo.get_by_id(member_id)
    if not member:
        raise ValueError("Hoi vien khong ton tai")

    # Validate fields moi
    if 'phone' in kwargs:
        if not re.match(r'^0\d{9}$', kwargs['phone']):
```

```
            raise ValueError("SDT khong hop le")

    # Cap nhat va luu
    for key, value in kwargs.items():
        setattr(member, key, value)
    member.update()  # Cap nhat updated_at
    return member_repo.update(member)
```

Listing 7: Cập nhật thông tin hội viên

6.4. Xóa (Delete) – Soft Delete

```
# Repository: soft delete + cascade
def delete(member_id):
    with get_db() as conn:
        now = datetime.now().isoformat()
        # Soft delete member
        conn.execute(
            "UPDATE members SET is_active=0, updated_at=? "
            "WHERE id=?", (now, member_id))
        # Cascade: huy subscriptions
        conn.execute(
            "UPDATE subscriptions SET is_active=0, "
            "status='cancelled', updated_at=? "
            "WHERE member_id=?", (now, member_id))
        # Cascade: huy trainer assignments
        conn.execute(
            "UPDATE trainer_assignments SET is_active=0, "
            "status='inactive', updated_at=? "
            "WHERE member_id=?", (now, member_id))
        # Hard delete notifications
        conn.execute(
            "DELETE FROM notifications WHERE user_id=?",
            (member_id,))
```

Listing 8: Xóa mềm hội viên (cascade)

7. Một số đoạn code tiêu biểu

7.1. Nhận diện khuôn mặt (Face ID)

Module Face ID sử dụng thư viện `face_recognition` (dựa trên dlib) để mã hóa và nhận diện khuôn mặt:

```
# app/face_id/face_encoder.py
import face_recognition
import numpy as np

class FaceEncoder:
    def encode_face(self, image):
        """Ma hoa khuon mat thanh vector 128 chieu"""
        # Detect vi tri khuon mat trong anh
        face_locations = face_recognition.face_locations(image)
```

```python
        if not face_locations:
            return None

        # Ma hoa thanh vector so (128-dimensional embedding)
        encodings = face_recognition.face_encodings(
            image, face_locations
        )
        return encodings[0] if encodings else None
```

Listing 9: Mã hóa khuôn mặt

```python
# app/face_id/face_recognizer.py
class FaceRecognizer:
    def __init__(self, tolerance=0.45):
        self.tolerance = tolerance  # Nguong tuong dong

    def recognize(self, face_encoding, known_encodings):
        """So sanh encoding voi database da biet"""
        for member_id, known_enc in known_encodings.items():
            # Tinh khoang cach Euclidean
            distance = face_recognition.face_distance(
                [known_enc], face_encoding
            )[0]

            if distance <= self.tolerance:
                confidence = 1.0 - distance
                return member_id, confidence

        return None, 0.0  # Khong nhan dien duoc
```

Listing 10: Nhận diện khuôn mặt khi điểm danh

Quy trình điểm danh:

1. Camera chụp ảnh khuôn mặt hội viên.
2. `FaceEncoder` mã hóa thành vector 128 chiều.
3. `FaceRecognizer` so sánh với database encoding đã lưu.
4. Nếu khoảng cách $\leq$ tolerance (0.45): nhận diện thành công $\rightarrow$ tự động check-in.
5. Lưu vào bảng `attendance` với độ tin cậy (confidence).

7.2. BaseModel và Soft Delete Pattern

```python
# app/models/base.py
import uuid
from datetime import datetime

class BaseModel:
    def __init__(self, *args, **kwargs):
        self.id = str(uuid.uuid4())       # UUID4 duy nhat
        self.created_at = datetime.now()  # Thoi diem tao
        self.updated_at = datetime.now()  # Thoi diem cap nhat
```

```python
        self.is_active = True              # Soft delete flag

    def update(self):
        """Cap nhat updated_at"""
        self.updated_at = datetime.now()

    def delete(self):
        """Xoa ao -- danh dau is_active = False"""
        self.is_active = False
        self.update()

    def to_dict(self):
        """Chuyen object thanh dict"""
        return {
            k: v.isoformat() if isinstance(v, datetime) else v
            for k, v in self.__dict__.items()
        }
```

Listing 11: BaseModel – lớp nền cho mọi model

7.3. Navigation Pattern – Monkey-patch trên page

```python
# app/main.py
def main(page: ft.Page):
    init_db()  # Khoi tao database

    def navigate(route):
        page.overlay.clear()
        page.controls.clear()
        page.on_search_change = None

        # Lazy import tranh circular dependency
        if route == "login":
            from gui.admin.login import LoginScreen
            page.add(LoginScreen(page))
        elif route == "dashboard":
            from gui.admin.dashboard import DashboardScreen
            page.add(DashboardScreen(page))
        elif route == "members":
            from gui.admin.members import MembersScreen
            page.add(MembersScreen(page))
        # ... cac route khac

        page.update()

    page.navigate = navigate  # Monkey-patch
    navigate("login")         # Bat dau tai login

ft.app(target=main)
```

Listing 12: Hệ thống điều hướng (routing)

7.4. GUI State với Dict Closures

Python closures không thể gán lại biến ngoài (nonlocal), nên dùng dict:

```python
def MembersScreen(page):
    # Dung dict de luu tru state (workaround closure)
    search_query = {"value": ""}
    current_filter = {"gender": "all", "status": "all"}

    def on_search(e):
        search_query["value"] = e.data  # Gan vao dict OK
        refresh_list()

    def on_filter_change(e):
        current_filter["gender"] = gender_dropdown.value
        refresh_list()

    # ... render UI
```

Listing 13: Quản lý state trong GUI

7.5. Đăng ký khuôn mặt (Face Registration)

```python
# app/face_id/face_register.py
class FaceRegistration:
    def register_member(self, member_id, images):
        """Dang ky khuon mat tu nhieu anh"""
        encodings = []
        for img in images:
            enc = self.encoder.encode_face(img)
            if enc is not None:
                encodings.append(enc)

        if not encodings:
            return False, "Khong phat hien khuon mat"

        # Lay encoding trung binh (robust hon)
        avg_encoding = np.mean(encodings, axis=0)

        # Luu vao file pickle
        self.save_encoding(member_id, avg_encoding)

        # Luu anh vao data/dataset/{member_id}/
        self.save_images(member_id, images)

        return True, "Dang ky thanh cong"
```

Listing 14: Đăng ký khuôn mặt cho hội viên

Dữ liệu Face ID được lưu tại:

- **Encoding**: `data/encodings/face_encodings.pkl` (file pickle chứa vector 128 chiều).
- **Ảnh gốc**: `data/dataset/{member_id}/*.jpg` (5–10 ảnh mỗi người).

Kết luận

Hệ thống **Gym Management System with Face ID** đã được xây dựng thành công với các đặc điểm nổi bật:

- **Kiến trúc phân tầng rõ ràng**: GUI → Service → Repository → Database, dễ bảo trì và mở rộng.
- **Tích hợp Face ID**: Nhận diện khuôn mặt tự động qua camera, nâng cao trải nghiệm người dùng.
- **2 ứng dụng riêng**: Admin và User, cùng chia sẻ backend.
- **10 bảng dữ liệu**: Quản lý đầy đủ thông tin hội viên, gói tập, thiết bị, HLV, điểm danh, thanh toán.
- **Soft delete**: Bảo toàn dữ liệu, hỗ trợ khôi phục.
- **Công nghệ hiện đại**: Python + Flet + SQLite + dlib/OpenCV.

Hệ thống hiện đạt khoảng **85% MVP**, có thể tiếp tục phát triển thêm các tính năng: thống kê nâng cao, xuất báo cáo PDF, tích hợp thanh toán điện tử, và ứng dụng di động.